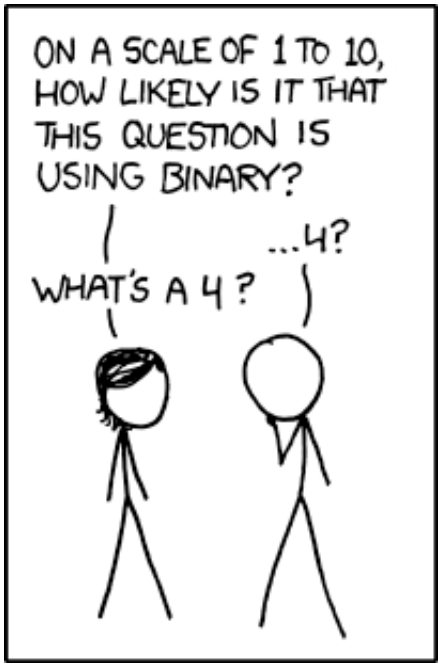
# Reference and Practice Handout

Lecture-04: Logical Operations

Computer Organization & Assembly Language Programming
CS 2160 || Fall 2026
Instructor: Himon Thakur




**1 to 10**
(Randall Munroe; CC BY-NC 2.5; https://xkcd.com/953/)

This document is a supplementary resource alongside course materials and class lectures. You are expected to go through and utilize it by yourself. You may read a section, then try solving the short exercises before moving on. Answers are provided at the end.

This handout will cover:

- Reading **unsigned** and **signed** (two's complement) integers.
- *Negating* a number, *one's* vs. *two's complement*, and *sign extension*.
- **Shifting** bits left and right, and when a shift means multiply or divide.
- Using **AND**, **OR**, **XOR**, and **NOT** to clear, set, flip, and pick out bits.

I used Claude Opus 5.5 to help me format the LaTeX code. All contents were produced, edited, or verified by me.

# 1. Signed and Unsigned Numbers

The hardware just stores bits. **Unsigned** treats every bit as a positive power of 2. **Two's complement** gives the top bit a *negative* weight. The same 32 bits can be a big positive number or a negative one.

## 1.1. Unsigned integers

$$x = x_{n-1}2^{n-1} + x_{n-2}2^{n-2} + \cdots + x_1 2^1 + x_0 2^0, \qquad \text{range: } 0 \text{ to } 2^n - 1$$

**Example 1: Reading `0000 ... 1011`$_2$**

$$1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 8 + 0 + 2 + 1 = 11_{10}$$

## 1.2. Two's complement (signed) integers

The only change: the top bit counts as **negative**.

$$x = -x_{n-1}2^{n-1} + x_{n-2}2^{n-2} + \cdots + x_1 2^1 + x_0 2^0, \qquad \text{range: } -2^{n-1} \text{ to } 2^{n-1} - 1$$

**Example 2: Reading `1111 1111 ... 1111 1100`$_2$ (32 bits)**

$$-1 \times 2^{31} + (2^{30} + \cdots + 2^2) = -2{,}147{,}483{,}648 + 2{,}147{,}483{,}644 = -4_{10}$$

| Landmark | 32-bit pattern | Value |
|---|---|---|
| zero | `0000 0000 ... 0000` | 0 |
| minus one | `1111 1111 ... 1111` | $-1$ |
| most negative | `1000 0000 ... 0000` | $-2^{31} = -2{,}147{,}483{,}648$ |
| most positive | `0111 1111 ... 1111` | $2^{31} - 1 = 2{,}147{,}483{,}647$ |

- Bit 31 is the **sign bit**: 1 = negative, 0 = non-negative.
- Non-negative numbers look the same in unsigned and two's complement.
- The range is lopsided: there is one more negative number than positive. So $-(-2^{31})$ **cannot be represented** in 32 bits.

**Exercise 1**

The 8-bit pattern `1111 1010` is in a register. What is its value as (a) unsigned, and (b) two's complement?

## 1.3. Negating a number: one's vs. two's complement

- **One's complement:** flip every bit ($0 \to 1$, $1 \to 0$). Simple, but it has two zeros ($+0$ = `0000`, $-0$ = `1111`), which complicates hardware.
- **Two's complement:** flip every bit, **then add 1**. One zero, and addition works the same for signed and unsigned. This is what RISC-V uses.

**Example 3: Negate +2 (32 bits)**

| | | |
|---|---|---|
| +2: | `0000 0000 ... 0000 0010` | |
| flip: | `1111 1111 ... 1111 1101` | (one's complement) |
| +1: | `1111 1111 ... 1111 1110` | $= -2$ (two's complement) |

**Exercise 2**

Write $-12$ as an 8-bit two's complement number, in binary and in hex.

### 1.4. Sign extension

To put a small number into a wider register *without changing its value*:

- **Signed:** copy the sign bit into every new bit on the left.
- **Unsigned:** fill the new bits with 0s (*zero extension*).

**Example 4: 8-bit → 16-bit**

+2: `0000 0010` → `0000 0000 0000 0010`
−2: `1111 1110` → `1111 1111 1111 1110`

In RISC-V: `lb` sign-extends the loaded byte; `lbu` zero-extends it.

**Exercise 3**

Memory holds the byte `0x80`. What 32-bit value (hex and decimal) does `lb` load? What does `lbu` load?

## 2. Logical Operations: Working on Bits, Not Numbers

Arithmetic treats a register as *one number*. Logical operations treat it as **32 separate bits** (or small groups of bits, called *fields*) and act on each one.

| Operation | C | Java | RISC-V |
|---|---|---|---|
| Shift left logical | << | << | `sll`, `slli` |
| Shift right logical (unsigned) | >> | >>> | `srl`, `srli` |
| Shift right arithmetic (signed) | >> | >> | `sra`, `srai` |
| Bit-by-bit AND | & | & | `and`, `andi` |
| Bit-by-bit OR | \| | \| | `or`, `ori` |
| Bit-by-bit XOR | ^ | ^ | `xor`, `xori` |
| Bit-by-bit NOT | ~ | ~ | `xori rd, rs, -1` |

**Note on C:** >> on an `unsigned` value is a logical shift. On a `signed` value it is an arithmetic shift on virtually every compiler (the C standard leaves negative values implementation-defined).

## 3. Shift Operations

A shift slides every bit left or right by some number of places. Bits that fall off the end are lost. The question is what fills the empty spots.

| Instruction | Empty spots filled with | Meaning for numbers |
|---|---|---|
| `sll` / `slli` (shift left logical) | 0s on the right | multiply by $2^i$ |
| `srl` / `srli` (shift right logical) | 0s on the left | divide by $2^i$ (*unsigned only*) |
| `sra` / `srai` (shift right arithmetic) | copies of the old sign bit | divide by $2^i$, rounding down (signed) |

`sll`, `srl`, `sra` take the shift amount from a register; the `-i` versions take it from the instruction.

**Color key used below:**

**opcode** **funct** (extra operation bits) **register** **immediate** (constant)

**Format of an immediate shift (RV32I):** the 12-bit immediate is split. The low 5 bits hold the shift amount (0–31). The top 7 bits act like `funct7` and tell `srli` (`0000000`) and `srai` (`0100000`) apart.

| funct7 | shamt | rs1 | funct3 | rd | opcode |
|---|---|---|---|---|---|
| 7 bits | 5 bits | 5 bits | 3 bits | 5 bits | 7 bits |

**Watch out**

**32-bit vs. 64-bit.** In RV64 (64-bit registers), the shift amount needs 6 bits (0–63), so the format becomes `funct6` (6 bits) + `shamt` (6 bits). The textbook slides show 64-bit registers; this handout uses 32-bit RV32, matching the rest of the course.

### 3.1. Shift left logical: multiply by a power of 2

**Example 5: `slli x11, x19, 4` with `x19` = 9**

```
x19:  00000000 00000000 00000000 00001001   = 9
x11:  00000000 00000000 00000000 10010000   = 144 = 9 × 2^4
```

Machine code: `0x00499593`.

**Practical use:** multiply by constants that are not powers of 2 by combining shifts. For example, $x \times 10 = x \times 8 + x \times 2 = (x \ll 3) + (x \ll 1)$.

**Exercise 4**

Write three RISC-V instructions that compute `a0` = 10× `a0` using shifts and an add (no multiply). Use `t0` and `t1` as scratch.

### 3.2. Shift right: logical vs. arithmetic

**Example 6: Shifting −9 right by 4**

```
        x19 = −9:  11111111 11111111 11111111 11110111
srai x11, x19, 4:  11111111 11111111 11111111 11111111   = −1
srli x11, x19, 4:  00001111 11111111 11111111 11111111   = 268,435,455
```

`srai` copies the sign bit (1) into the empty spots, so the result stays negative.
`srli` fills with 0, so the result is a huge positive number.

**Watch out**

`srai` by $i$ divides by $2^i$ but **rounds toward** $-\infty$. C's `/` **rounds toward 0**. So `srai` of −9 by 4 gives −1, while −9/16 in C gives 0. Compilers add a small fix-up when they turn signed division into a shift.

**Exercise 5**

`x5` holds −100 (`0xFFFFFF9C`). Give the result of (a) `srai x6, x5, 2` and (b) `srli x7, x5, 2`, in hex and decimal.

## 4. AND, OR, XOR, NOT

Each of these works on **one bit position at a time**: bit 0 with bit 0, bit 1 with bit 1, and so on. The second operand is usually a **mask**: a pattern that says which bits you want to touch.

| $a$ | $b$ | $a$ AND $b$ | $a$ OR $b$ | $a$ XOR $b$ |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 1 | 0 | 1 | 1 |
| 1 | 0 | 0 | 1 | 1 |
| 1 | 1 | 1 | 1 | 0 |

| Goal | Operation | Why it works |
|---|---|---|
| **Clear** bits (force to 0) | AND with 0 in those spots | $x$ AND $0 = 0$, $x$ AND $1 = x$ |
| **Keep / pick out** bits | AND with 1 in those spots | everything else becomes 0 |
| **Set** bits (force to 1) | OR with 1 in those spots | $x$ OR $1 = 1$, $x$ OR $0 = x$ |
| **Flip** bits | XOR with 1 in those spots | $x$ XOR $1 =$ NOT $x$, $x$ XOR $0 = x$ |
| **Flip all** bits (NOT) | XOR with all 1s ($-1$) | every bit is flipped |

## 4.1. Worked examples (32-bit)

For all three, `x10` = `0x00000DC0` and `x11` = `0x00003C00`. Only the low 16 bits are shown; the upper 16 are all 0.

**Example 7: `and x9, x10, x11`: keep only the bits where the mask is 1**

```
x10:  0000 1101 1100 0000
x11:  0011 1100 0000 0000
-------------------------
 x9:  0000 1100 0000 0000   = 0x00000C00
```

**Example 8: `or x9, x10, x11`: turn on every bit that is 1 in either**

```
x10:  0000 1101 1100 0000
x11:  0011 1100 0000 0000
-------------------------
 x9:  0011 1101 1100 0000   = 0x00003DC0
```

**Example 9: `xor x9, x10, x12` with `x12` = $-1$ (all 1s): NOT**

```
x10:  0000 0000 0000 0000 0000 1101 1100 0000
x12:  1111 1111 1111 1111 1111 1111 1111 1111
---------------------------------------------
 x9:  1111 1111 1111 1111 1111 0010 0011 1111   = 0xFFFFF23F
```

RISC-V has no NOT instruction. The assembler's `not rd, rs` is just `xori rd, rs, -1`.

## 4.2. Practical bit tricks

**Example 10: Changing letter case**

In ASCII, upper- and lowercase letters differ only in **bit 5** (value `0x20`): `'a'` = `0x61` = `0110 0001`, `'A'` = `0x41` = `0100 0001`.

```
andi  a0, a0, 0xDF    # clear bit 5  -> uppercase   ('a' -> 'A')
ori   a0, a0, 0x20    # set bit 5    -> lowercase   ('Q' -> 'q')
xori  a0, a0, 0x20    # flip bit 5   -> swap case   ('m' -> 'M')
```

(Works for letters only. These assume `a0` holds a single ASCII letter.)

**Example 11: Pulling a field out of an instruction word**

Every RISC-V instruction is itself stored as a 32-bit number made of fields (more on this in Lecture 07). For example, `add x9, x20, x21` is stored as `0x015A04B3`, and its destination register number `rd` sits in bits 11–7. To extract it: **shift** the field down to bit 0, then **AND** with a mask of five 1s (`0x1F`).

```
srli  t0, a0, 7       # move bits 11-7 down to bits 4-0
andi  t0, t0, 0x1F    # keep only those 5 bits  -> t0 = 9
```

This "shift then mask" pattern is how decoders, network code, and file parsers read packed fields.

**Exercise 6**

`x10` = `0xCAFEF00D`. Give the result of each:
(a) `andi x6, x10, 0xFF` (b) `ori x7, x10, 0xF0` (c) `xori x8, x10, 0x0F`

**Exercise 7**

`a0` holds an instruction word. Write two instructions that put its `rs1` field (bits 19–15) in `t0`. What value do you get for `0x015A04B3`?

## 5. Summary: Arithmetic & Logical Instructions

| Format | Computation | Notes |
|---|---|---|
| `add rd, rs1, rs2` | `rd = rs1 + rs2` | |
| `sub rd, rs1, rs2` | `rd = rs1 - rs2` | |
| `sll rd, rs1, rs2` | `rd = rs1 << rs2` | fills 0s on the right |
| `srl rd, rs1, rs2` | `rd = rs1 >> rs2` (logical) | fills 0s on the left |
| `sra rd, rs1, rs2` | `rd = rs1 >> rs2` (arithmetic) | copies the sign bit |
| `and rd, rs1, rs2` | `rd = rs1 & rs2` | |
| `or rd, rs1, rs2` | `rd = rs1 \| rs2` | |
| `xor rd, rs1, rs2` | `rd = rs1 ^ rs2` | |
| `slt rd, rs1, rs2` | `rd = (rs1 < rs2) ? 1 : 0` | signed compare; `sltu` is unsigned |

- Every one of these has an immediate form (`addi`, `slli`, `andi`, ...) **except** `sub`. To subtract a constant, add a negative one: `addi x5, x5, -4`.
- Immediates are 12-bit two's complement, sign-extended to 32 bits before use.
- Handy pseudo-instructions: `not rd, rs` = `xori rd, rs, -1`; `neg rd, rs` = `sub rd, x0, rs`; `ret` = `jalr x0, 0(ra)`.
- Multiply and divide are not in the base RV32I set; they come from the "M" extension.

| R-type | funct7 | funct3 | opcode | I-type | imm[11:5] | funct3 | opcode |
|---|---|---|---|---|---|---|---|
| `add` | 0000000 | 000 | 0110011 | `addi` | - | 000 | 0010011 |
| `sub` | 0100000 | 000 | 0110011 | `slli` | 0000000 | 001 | 0010011 |
| `sll` | 0000000 | 001 | 0110011 | `slti` | - | 010 | 0010011 |
| `slt` | 0000000 | 010 | 0110011 | `sltiu` | - | 011 | 0010011 |
| `sltu` | 0000000 | 011 | 0110011 | `xori` | - | 100 | 0010011 |
| `xor` | 0000000 | 100 | 0110011 | `srli` | 0000000 | 101 | 0010011 |
| `srl` | 0000000 | 101 | 0110011 | `srai` | 0100000 | 101 | 0010011 |
| `sra` | 0100000 | 101 | 0110011 | `ori` | - | 110 | 0010011 |
| `or` | 0000000 | 110 | 0110011 | `andi` | - | 111 | 0010011 |
| `and` | 0000000 | 111 | 0110011 | | | | |

**Exercise 8**

Let `a0` = $-1$ and `a1` = 1. What does `slt t0, a0, a1` produce? What does `sltu t1, a0, a1` produce? Why are they different?

## 6. Absolute Value Without a Branch (Bit Twiddling)

```
int abs(int v) {
    int mask = v >> 4 * 8 - 1;       // = v >> 31
    return (v + mask) ^ mask;
}
```

### 6.1. Step 1: read the shift amount correctly

C operator precedence decides the grouping: `*` is applied first, then `-`, then `>>`.

$$\texttt{v >> 4 * 8 - 1} = \texttt{v >> ((4 * 8) - 1)} = \texttt{v >> 31}$$

($4 \times 8 = 32$ is the number of bits in an `int`.)

### 6.2. Step 2: what is `mask`?

An *arithmetic* shift right by 31 copies the sign bit into every position:

- $v \geq 0$: `mask` = `0000...0000` $= 0$.
- $v < 0$: `mask` = `1111...1111` $= -1$.

### 6.3. Step 3: why `(v + mask) ^ mask` works

- If $v \geq 0$: $(v + 0) \oplus 0 = v$. Nothing changes.
- If $v < 0$: $(v - 1) \oplus (-1) = \text{NOT}(v - 1)$. And $\text{NOT}(v - 1) = -v$, because two's complement says $-x = \text{NOT}(x) + 1$, so $\text{NOT}(v - 1) = -(v - 1) - 1 = -v$.

**Example 12: Trace with $v = -5$ (8 bits shown)**

| Step | Bits | Value |
|---|---|---|
| $v$ | 1111 1011 | $-5$ |
| `mask = v >> 7` | 1111 1111 | $-1$ |
| `v + mask` | 1111 1010 | $-6$ |
| `(v + mask) ^ mask` | 0000 0101 | 5 |

### 6.4. In RISC-V

The argument $v$ arrives in `a0`, and the result is returned in `a0`.

```
abs:
    srai  t0, a0, 31      # t0 = mask (all 0s or all 1s)
    add   a0, a0, t0      # a0 = v + mask
    xor   a0, a0, t0      # a0 = (v + mask) ^ mask
    ret                   # return a0
```

Four instructions and **no branch**. Branches can slow a pipelined processor when it guesses the direction wrong, so branch-free tricks like this are common in performance-critical code.

**Watch out**

**The one input that breaks it:** $v = -2^{31}$ (`INT_MIN`). Its positive version $2^{31}$ does not fit in 32 bits (Section 1.2), so `abs` returns `INT_MIN` unchanged. In C, the `v + mask` step for this input is also signed overflow, which is undefined behavior.

**Exercise 9**

Trace the RISC-V `abs` code for `a0` $= -8$. Write `t0` and `a0` after each instruction (hex is fine).

## 7. Absolute Value With a Branch

```
int abs(int v) {
    return (v < 0) ? -(unsigned int)v : v;
}
```

The ternary operator `cond ? A : B` means "if `cond` then `A` else `B`". The `(unsigned int)` cast makes the negation well-defined in C even for `INT_MIN`.

```
abs:
        slt   t0, a0, zero    # t0 = 1 if a0 < 0, else 0
        beq   t0, zero, .L1   # if t0 == 0 (v is NOT negative), skip to .L1
        sub   a0, zero, a0    # v was negative: a0 = 0 - a0 = -v
.L1:    ret                   # return a0
```

**New instruction: `slt` (set less than).** `slt rd, rs1, rs2` writes 1 into `rd` if `rs1` $<$ `rs2` (signed), otherwise 0. It turns a comparison into a number you can branch on or compute with.

## 7.1. Walking both paths

| **Instruction** | $v = 7$ | $v = -7$ |
|---|---|---|
| `slt t0, a0, zero` | `t0` $= 0$ | `t0` $= 1$ |
| `beq t0, zero, .L1` | taken: jump to `.L1` | not taken: fall through |
| `sub a0, zero, a0` | (skipped) | `a0` $= 7$ |
| `.L1: ret` | returns 7 | returns 7 |

A **label** like `.L1` is just a name for an address. It takes no space in the program; writing it on its own line or on the same line as the instruction makes no difference.

> **Two ways, same answer.** The branch version is easier to read. The bit-twiddling version has no branch. Both return `INT_MIN` for `INT_MIN`.

# Recap

**Remember these**

1. Two's complement: the top bit is worth $-2^{n-1}$. Range $-2^{n-1}$ to $2^{n-1}-1$.
2. Negate = flip all bits, then add 1. Flipping alone is one's complement (two zeros, not used).
3. Sign extension copies the sign bit (`lb`); zero extension fills 0s (`lbu`).
4. `slli` by $i$ = multiply by $2^i$. `srli` = unsigned divide. `srai` = signed divide, rounding down.
5. AND clears / picks out bits. OR sets bits. XOR flips bits. NOT = `xori rd, rs, -1`.
6. To read a field: shift it down to bit 0, then AND with a mask of 1s.
7. No `subi` (use `addi` with a negative), no NOT, no multiply in base RV32I.
8. `slt` turns a comparison into 0 or 1. `sltu` does the same for unsigned values.
9. `abs` can be done branch-free with `srai`/`add`/`xor`, or with `slt`/`beq`/`sub`. Neither can fix `INT_MIN`.

# Answer key for Exercises

**Ex. 1.** (a) Unsigned: $128+64+32+16+8+2=250$. (b) Signed: $-128+122=-6$.

**Ex. 2.** $12$ = `0000 1100` → flip `1111 0011` → $+1$ → `1111 0100` = `0xF4`.

**Ex. 3.** `0x80` = `1000 0000`, sign bit 1. `lb`: `0xFFFFFF80` $=-128$. `lbu`: `0x00000080` $=128$.

**Ex. 4.** `slli t0, a0, 3` (`t0` $=8v$) `slli t1, a0, 1` (`t1` $=2v$) `add a0, t0, t1` (`a0` $=10v$).

**Ex. 5.** (a) `srai`: `0xFFFFFFE7` $=-25$ ($-100/4$ exactly).
(b) `srli`: `0x3FFFFFE7` = 1,073,741,799 (the sign bit was replaced by 0s).

**Ex. 6.** (a) `0x0000000D` (only the low byte survives). (b) `0xCAFEF0FD` (bits 7–4 forced to 1). (c) `0xCAFEF002` (bits 3–0 flipped: `D` = 1101 → 0010).

**Ex. 7.** `srli t0, a0, 15` then `andi t0, t0, 0x1F`. For `0x015A04B3`: `t0` = 20 (`add x9, x20, x21` has rs1 = 20).

**Ex. 8.** `slt`: signed, $-1<1$ → `t0` = 1. `sltu`: unsigned, `0xFFFFFFFF` = 4,294,967,295 is *not* less than 1 → `t1` = 0. Same bits, different comparison rule.

**Ex. 9.**

| | | |
|---|---|---|
| after `srai t0, a0, 31` | `t0` = `0xFFFFFFFF` ($-1$) | `a0` = `0xFFFFFFF8` ($-8$) |
| after `add a0, a0, t0` | `t0` = `0xFFFFFFFF` | `a0` = `0xFFFFFFF7` ($-9$) |
| after `xor a0, a0, t0` | `t0` = `0xFFFFFFFF` | `a0` = `0x00000008` (8) |

# Student Practice Worksheet

## Part 1: Signed & Unsigned

**Q1.** Give the range of a 16-bit number as
(a) unsigned: _____ (b) two's complement: _____

**Q2.** For +5 in 8 bits, write the one's complement: _____ and the two's complement: _____. Which one equals −5?

**Q3.** Sign-extend the 4-bit value `1011` to 8 bits: _____. What decimal value does it represent? _____

## Part 2: Shifts & Masks

**Q4.** `x5` = `0x0000000F`. What is `x6` after `slli x6, x5, 3`? `0x`_____ (= _____ in decimal)

**Q5.** `x5` = `0x80000000`. Give the result of each in hex:

- `srai x6, x5, 4`: `0x`_____
- `srli x7, x5, 4`: `0x`_____

**Q6.** `x5` = `0x00000071`. Write one instruction for each step, applied in order, and give `x5` at the end:

(a) set bit 3 (b) clear bit 0 (c) flip bit 7

**Q7.** `a0` holds an instruction word. Write two instructions that put its `funct3` field (bits 14–12) in `t0`. What do you get for `0x020B2483` (`lw x9, 32(x22)`)?

## Part 3: Putting It Together

**Q8.** RISC-V has no NOT instruction. Explain how to get one, and why XOR with −1 works.

**Q9.** Rewrite the branching `abs` using a single `bge` instead of `slt` + `beq`. (`bge rs1, rs2, L` jumps to `L` if `rs1` ≥ `rs2`, signed.) How many instructions does it use?

**Q10.** What do *both* `abs` versions return for `INT_MIN` (`0x80000000`)? Why can't any 32-bit `abs` do better?

# Step-by-Step Solutions & Explanations

**Q1: 16-bit ranges**

- Unsigned: 0 to $2^{16} - 1 = 65{,}535$.
- Two's complement: $-2^{15} = -32{,}768$ to $2^{15} - 1 = 32{,}767$.

**Q2: One's vs. two's complement of $+5$**

- $+5 =$ `0000 0101`.
- One's complement (flip): `1111 1010`. Two's complement (flip, $+1$): `1111 1011`.
- In two's complement arithmetic (what RISC-V uses), $-5 =$ `1111 1011`. The one's complement pattern reads as $-6$.

**Q3: Sign-extend `1011`**

- The sign bit is 1, so copy it into the four new bits: `1111 1011`.
- Value: $-8 + 2 + 1 = -5$ (4-bit) $= -128 + 123 = -5$ (8-bit). The value is unchanged.

**Q4: `slli x6, x5, 3`**

- `1111` → `111 1000`: `0x00000078` $= 120 = 15 \times 2^3$.

**Q5: Shifting `0x80000000` right by 4**

- `srai`: the sign bit (1) fills the top 4 bits → `0xF8000000` ($-2^{31}/16 = -2^{27}$).
- `srli`: 0s fill the top 4 bits → `0x08000000` ($2^{31}/16 = 2^{27}$, read as unsigned).

**Q6: Set, clear, flip**

- (a) `ori x5, x5, 0x8` → `0x00000079`.
- (b) `andi x5, x5, -2` ($-2 = \ldots 1111\,1110$, a 0 only in bit 0) → `0x00000078`.
- (c) `xori x5, x5, 0x80` → `0x000000F8`.

**Q7: Extract funct3**

- `srli t0, a0, 12` then `andi t0, t0, 0x7` (three 1s, because funct3 is 3 bits).
- For `0x020B2483`: `t0` $= 2 =$ `010`, the funct3 of `lw`.

**Q8: NOT Answer:** Use `xori rd, rs, -1` (the assembler accepts `not rd, rs` for this). The immediate $-1$ is sign-extended to all 32 ones, and $x \oplus 1 =$ NOT $x$ for every bit, so every bit flips.

**Q9: `abs` with `bge`**

```
abs:
        bge   a0, zero, .L1   # if v >= 0, return v unchanged
        sub   a0, zero, a0    # otherwise a0 = -v
.L1:    ret
```

Three instructions (one fewer), because `bge` compares and branches in one step.

**Q10: `abs(INT_MIN)` Answer:** Both return `0x80000000` $= -2^{31}$. The true answer $+2^{31}$ is one more than the largest 32-bit signed value ($2^{31} - 1$), so no 32-bit signed result can hold it.